LR-GoWU: Learned-Return Go-With-Uncertainty

Pseudo-algorithms (iteration v2, commit `d33407c`)

Idea. GoWU [1] redistributes particles by *restoring* emulator states ($\textsc{Reset}(p_{\text{target}}, p_{\text{source}})$). LR-GoWU keeps GoWU's winner selection, group consolidation, global synchronization and rollback, but never restores a state: every episode starts at $s_0 = \mathcal{E}.\textsc{reset}()$ and a learned goal-conditioned *return policy* π_R walks to the selected landmark. Exploration with GoWU's particle policy starts only after this *handoff*. The search tree of checkpoints becomes an archive of *landmarks*, one per discretized cell. The archive supplies hindsight demonstrations that π_R imitates. The output is a deployable policy (run π_R from reset), not only a trajectory. The primed numbers mark the counterpart of the paper's algorithm.

GoWU (paper)	**LR-GoWU (here)**
particle p_i on a simulator checkpoint	episode slot i, always started from s_0
$\textsc{Reset}(p_i, v)$: copy a checkpoint into p_i	return phase: π_R walks from s_0 to landmark g (Alg. 1′, l. 5–18)
one particle's life from one reset to its next (several outer steps)	one RunEpisode (Alg. 1′)
one outer step: one held random action for $T_i \sim \text{Unif}(T_{\min}, T_{\max})$ inner steps	one hold segment of the explore phase, $h \sim \text{Unif}\{5..15\}$ steps (Alg. 1′, l. 22)
stop the rollout as soon as R increases (Alg. 1, l. 8)	*not used*: exploration continues after a reward
$\mathcal{T}.\textsc{AddChild}(p_i)$ at the end of each outer step	UpdateArchive after every Alive step (Alg. 2′)
reset Dead particles to the winner	a dead slot's next goal is the group winner (Alg. 4′, l. 17)
rollback when *all* particles of a group die	rollback when returns to one landmark keep failing (Alg. 3′)
$U \leftarrow p_i.\textsc{Step}(1, U)$: U is read and trained during rollouts	U is read inside UpdateArchive and Winner; every frame goes into the RND replay in Step; the RND predictor is trained once per unroll (Alg. 4′, l. 30)

Terms and definitions

Objects.

cell(s)	A function from a state to a discrete key, read from the emulator RAM: cell(s) = (level, room, #keys held, $\lfloor x/16 \rfloor$, $\lfloor y/16 \rfloor$), where (x, y) is the player's pixel position. States with the same key are treated as the same place.
Landmark c	The record kept for one cell (we name the landmark by its cell key, so "cell(s) = c" means s is in landmark c's cell). It stores: f_c, an 84×84 grey frame of the stored state; R_c, the episode's cumulative reward at the moment that state was seen; U_c, the uncertainty of f_c; u_c, the parent landmark; d_c, the depth ($d_{u_c} + 1$); its demos; and its return statistics. A cell's landmark can be overwritten by a better state but is never deleted.
Archive $\mathcal{A}$	The set of all landmarks. The root c_0 = cell(s_0) is the start cell with $R_{c_0} = 0$. Parent pointers make $\mathcal{A}$ a tree.
$U(f)$	A number: the uncertainty of frame f, measured as the RND prediction error (squared error between a trained predictor network and a fixed random target network). Frames seen often have low error. One RND is shared by all slots and is trained throughout the run, so a stored U_c goes stale; it is recomputed ("re-scored") as $U(f_c)$ whenever it is compared.
$\succ^{\text{lex}}$, lex-argmax	As in the paper. $(b, c) \succ^{\text{lex}} (b', c')$ iff $b > b'$, or $b = b'$ and $c > c'$. lex-argmax$_{c \in S}(R_c, U_c)$ returns the landmark in the set S with the highest R_c, breaking ties by the highest re-scored U_c.
Dead / Alive	A step is Dead if a life is lost, the game is over, or the exploit guard fires (a RAM check for the middle-room key bug; the bug's reward is set to 0). A Dead step ends the episode. Every other step is Alive.
Reach(s, R, g)	Returns true or false. s is the current state, R the episode's cumulative reward so far (the sum of rewards since reset, not the one-step reward r), and g a landmark. It is true iff cell(s) = g *and* $R \geq R_g$: the agent stands in g's cell having collected at least the reward that g was recorded with. Being in the right cell with less reward (for example, having skipped a jewel that g's route collected) does not count.
Anc$_k(g)$	The landmark reached by following parent pointers k times from g (Anc$_1(g) = u_g$). If the tree above g has fewer than k levels it returns the root. Anc$_{\leq 32}(g)$ is the list of g's 32 nearest ancestors.
Demo of c	A recorded action sequence that is known to reach c from reset: the prefix $\tau[0{:}t]$ of some episode, cut at the first step t where that episode was in c's cell with $R \geq R_c$. Each landmark keeps its best 4 demos ranked by R (higher first), then length (shorter first). π_R imitates them.
Return statistics of g	W_g: list of the last ≤64 return attempts to g, each 1 (reached) or 0 (timed out, stalled or died). succ(g): mean of W_g (0 if empty). ℓ_g: running average (EMA, weight 0.1) of how many steps successful returns took. Counters: n_g^{tgt}, episodes that chose g as goal; n_g^{reach}, returns that reached g; n_g^{hand}, handoffs at g; n_g^{qd}, handoffs followed by death within 8 steps ("quick death").
Eligible(g)	Returns true or false: may g be used as a goal? The root always may. Another g is *ineligible* if it looks unreachable ($n_g^{\text{tgt}} \geq 32$, $n_g^{\text{reach}} = 0$ and no demo to learn from), or doomed (at least 16 handoffs and more than 90% of them died within 8 steps). We also write Eligible for the set of eligible landmarks.
Fallback(g)	The first eligible landmark in the list $g, \text{Anc}_1(g), \text{Anc}_2(g), \ldots$; the root if none is.
Winner(m)	Group m's current winner: lex-argmax$_{c \in \{g_m\} \cup (I_m \cap \text{Eligible})}(R_c, U_c)$.
Slots and groups	$E = 128$ slots, each running one episode at a time, all stepped in lockstep. They form $M = 4$ groups $G_1, \ldots, G_M$ of $N = 32$ slots ($m(i)$ is slot i's group). Group m keeps a goal g_m, the set I_m of landmarks its slots improved since its last consolidation, an episode counter n_m and a consolidation target K_m. Slot i keeps ν_i, the goal of its next episode.
π_R, V_R	The return policy and its value function: one goal-conditioned actor-critic (separate IMPALA encoders). Its input at step t is an image x_t (the last 4 frames, the frame at the last reward, and the goal's frame f_g: 6 channels) and a context vector κ_t = (32 bits: which of Anc$_{\leq 32}(g)$ this episode has visited; clip($(R_g - R)/1000, 0, 5$), the reward still missing; $t/5000$, the time used).

Symbols inside the algorithms.

s, f, r, R, t, τ	Current state, its frame, last one-step reward, cumulative reward since reset, steps since reset, and the episode's trajectory $(s_0, a_1, s_1, \ldots)$.
$\textsc{Step}(a)$	One environment step with all its bookkeeping: $s, r, \mathit{dead} \leftarrow \mathcal{E}.\textsc{step}(a)$; $R \leftarrow R + r$; $t \leftarrow t + 1$; append (a, s) to τ; add f to the RND replay (the frames U is trained on).
I, J	I: landmarks this episode created or replaced so far. J: those created or replaced by the current step (empty or one landmark).
t_{prog}	Last step at which the episode entered a cell it had not visited before ("progress"). Drives the return stall test and β_t.
β_t	Temperature on π_R's logits (an entropy boost when stuck): $\beta_t = 1+(0.01 \max(0,\, t-t_{\text{prog}}-100))^2$. It is 1 until 100 steps pass without progress, then grows, flattening the action distribution.
L, $\mu L + b$	Expected return length for g and the return time limit derived from it.
handed, t_h	*handed* is true if this episode reached g (or started in it); t_h is the step at which that happened.
t_0, t_{imp}	Step at which the explore phase began; last step at which this episode improved a landmark.
a_{hold}, h	The held random action and the number of steps it is still held. $\mathcal{A}_{\text{act}}$ is the set of 18 Atari actions.
r_t^R	Reward used to train π_R with PPO (return phase only); the game reward r enters it only through $0.1\,\text{sign}(r)$.

Algorithm 1′ RunEpisode (one episode of slot i: return to g, then explore)

input: goal landmark $g \in \mathcal{A}$, return policy π_R, uncertainty U
params: return time limit $(\mu, b) = (2, 200)$, return stall 500, explore budget 1000, explore stall 300, episode cap $T_{\max} = 5000$, hold length $[h_{\min}, h_{\max}] = [5, 15]$, $\epsilon = 0.2$, quick death 8

$s \leftarrow \mathcal{E}.\text{RESET}()$; $R \leftarrow 0$; $t \leftarrow 0$; $\tau \leftarrow (s)$; $I \leftarrow \emptyset$; *handed* $\leftarrow$ false; $n_g^{\text{tgt}} \leftarrow n_g^{\text{tgt}} + 1$ // no restore
if cell$(s) = g$ and $R_g \le 0$ **then** // goal is the start cell (e.g. root)
  $n_g^{\text{reach}} \leftarrow n_g^{\text{reach}} + 1$; $n_g^{\text{hand}} \leftarrow n_g^{\text{hand}} + 1$; *handed* $\leftarrow$ true; $t_h \leftarrow 0$; **goto** line 20
/* Return phase: replaces Reset(p_i, v) by walking to g */
$L \leftarrow \ell_g$ if defined, else the length of g's best demo, else 1000
loop // each pass is one step; Step advances t
  $a_t \sim \pi_R(\cdot \mid x_t, \kappa_t)$ at temperature β_t
  dead $\leftarrow$ Step(a_t)
  if not *dead* **then** $I \leftarrow I \cup$ UpdateArchive(s, R, τ, i) // Algorithm 2′
  $r_t^R \leftarrow \mathbf{1}[\text{Reach}(s, R, g) \wedge \neg \textit{dead}] + 0.1 \cdot \#\{\text{ancestors in Anc}_{\le 32}(g) \text{ first visited at } t\} + 0.1\,\text{sign}(r)$
  if *dead* **then**
    RecordReturn$(g, 0)$; **return** (τ, I, Dead) // Algorithm 3′
  else if Reach(s, R, g) **then** // handoff: reached g
    RecordReturn$(g, 1)$; $\ell_g \leftarrow 0.9\,\ell_g + 0.1\,t$; $n_g^{\text{hand}} \leftarrow n_g^{\text{hand}} + 1$; *handed* $\leftarrow$ true; $t_h \leftarrow t$; **break**
  else if $t \ge \mu L + b$ **or** $t - t_{\text{prog}} \ge 500$ **or** $t \ge T_{\max}$ **then**
    RecordReturn$(g, 0)$ // failed: explore from here anyway
    if $t \ge T_{\max}$ **then return** (τ, I, Alive)
    break
/* Explore phase: GoWU's particle policy (held random action) */
$t_0 \leftarrow t$; $t_{\text{imp}} \leftarrow t$; $h \leftarrow 0$ // $\approx$ a run of Alg. 1 outer steps, l. 4--12
repeat
  if $h = 0$ **then** $a_{\text{hold}} \sim \text{Unif}(\mathcal{A}_{\text{act}})$; $h \sim \text{Unif}\{h_{\min}, \ldots, h_{\max}\}$
  $a_t \leftarrow a_{\text{hold}}$; $h \leftarrow h - 1$; with probability ϵ: $a_t \sim \text{Unif}(\mathcal{A}_{\text{act}})$
  dead $\leftarrow$ Step(a_t)
  if *dead* **then**
    if *handed* **and** $t - t_h \le 8$ **then** $n_g^{\text{qd}} \leftarrow n_g^{\text{qd}} + 1$ // feeds the doomed-landmark gate
    return (τ, I, Dead)
  $J \leftarrow$ UpdateArchive(s, R, τ, i); $I \leftarrow I \cup J$; **if** $J \neq \emptyset$ **then** $t_{\text{imp}} \leftarrow t$
until $t - t_0 \ge 1000$ **or** $t - t_{\text{imp}} \ge 300$ **or** $t \ge T_{\max}$
return (τ, I, Alive)

Algorithm 2′ UpdateArchive (every Alive step; replaces $\mathcal{T}$.AddChild)

input: state s (frame f), episode reward R, trajectory $\tau = \tau[0{:}t]$, slot i in group m
params: U hysteresis $\eta = 0.1$, demos per landmark 4, demo store cap 12 GB

$c \leftarrow \text{cell}(s)$; $c_{\text{prev}} \leftarrow \text{cell}(s_{t-1})$
if $c \in \mathcal{A}$ **and** $R = R_c$ **then** $U_c \leftarrow U(f_c)$ // re-score the incumbent with the current RND
if $c \notin \mathcal{A}$ **or** $R > R_c$ **or** $(R = R_c$ **and** $U(f) > (1 + \eta)\,U_c)$ **then** // $(R, U) \succ^{\text{lex}} (R_c, U_c)$ with hysteresis
  if $c \notin \mathcal{A}$ **then**
    $u_c \leftarrow c_{\text{prev}}$ if $c_{\text{prev}} \neq c$, else none; $d_c \leftarrow d_{u_c} + 1$ (or 0) // parent fixed at creation
  else if $R > R_c$ **then**
    drop c's demos, W_c and ℓ_c // old routes no longer earn R_c
  $(f_c, R_c, U_c) \leftarrow (f, R, U(f))$; $J \leftarrow \{c\}$; $I_m \leftarrow I_m \cup \{c\}$ // I_m grows live, not at episode end
else
  $J \leftarrow \emptyset$
/* Hindsight demonstration */
if this is the first step of τ in c with $R \ge R_c$ **and** $t > 0$ **then**
  offer $\tau[0{:}t]$ as a demo of c; keep the top 4 by (R desc., t asc.) // only first demos once store is full
return J

Algorithm 3′ RecordReturn (after every return attempt; failure recovery via rollback)

input: landmark g, outcome $o \in \{0, 1\}$
params: window $w = 64$, success threshold $\rho = 0.05$, rollback range $[k_{\min}, k_{\max}] = [1, 3]$

append o to W_g (keep the last w); **if** $o = 1$ **then** $n_g^{\text{reach}} \leftarrow n_g^{\text{reach}} + 1$
if $|W_g| = w$ **and** $\text{succ}(g) < \rho$ **then** // ≈ Alg. 1, l. 14--17
  for each group m with $g_m = g$ **do**
    $k \sim \text{Unif}\{k_{\min}, \ldots, k_{\max}\}$; $g_m \leftarrow \text{Anc}_k(g)$; $\nu_j \leftarrow g_m$ for all $j \in G_m$ // see Remark 1
  $W_g \leftarrow \emptyset$ // cleared even if no group targets g

Algorithm 4′ LR-GoWU (replaces GoWU's population loop)

input: environment $\mathcal{E}$, $E = 128$ slots in $M = 4$ groups
params: consolidation $[K_{\min}, K_{\max}] = [16, 32]$ episodes, mixing $p_{\text{mix}} = 0.1$, unroll $T = 128$

Initialize $\mathcal{A} \leftarrow \{c_0\}$ with $(f_{c_0}, 0, U(f_{c_0}))$; π_R, V_R; shared RND U and its frame replay
$g_m \leftarrow c_0$, $I_m \leftarrow \emptyset$, $n_m \leftarrow 0$, $K_m \sim \text{Unif}\{K_{\min}..K_{\max}\}$ for all m; $\nu_i \leftarrow g_{m(i)}$ for all i
loop // all E slots step in lockstep
  for T steps **do**
    for each slot i in group m **in parallel do**
      if slot i has no running episode **then** // episode start
        $g \leftarrow \nu_i$; **if not** Eligible(g) **then** $g \leftarrow$ Fallback(g_m) // falls back from the group goal
        start RunEpisode(g) in slot i // Algorithm 1′
      advance slot i's episode by one environment step
      if the episode is still running **then continue**
      /* Episode end with (τ, I, status): choose the next goal */
      if $\text{Unif}(0, 1) < p_{\text{mix}}$ **then**
        $\nu_i \sim \{c \in \mathcal{A} : \text{Eligible}(c)\}$ with probability $\propto 1/\sqrt{n_c^{\text{tgt}} + 1}$ // count-based mixing
      else if status = Alive **and** $I \cap \text{Eligible} \neq \emptyset$ **then**
        $\nu_i \leftarrow \text{lex-argmax}_{c \in I \cap \text{Eligible}}(R_c, U_c)$ // survivor continues; ≈ Alg. 1, l. 10--11
      else
        $\nu_i \leftarrow$ Winner(m) // ≈ Alg. 1, l. 23
      $n_m \leftarrow n_m + 1$
      if $n_m \geq K_m$ **then** // group consolidation; ≈ Alg. 1, l. 29--30
        $g_m \leftarrow$ Winner(m); $\nu_j \leftarrow g_m$ for all $j \in G_m$ // see Remark 1
        $I_m \leftarrow \emptyset$; $n_m \leftarrow 0$; $K_m \sim \text{Unif}\{K_{\min}..K_{\max}\}$
        $g^\star \leftarrow \text{lex-argmax}_{k \leq M}(R_{g_k}, U_{g_k})$ // global sync; ≈ Alg. 2, l. 8--10, 16
        for each group k with $R_{g_k} < R_{g^\star}$ **do**
          $g_k \leftarrow g^\star$; $\nu_j \leftarrow g^\star$ for all $j \in G_k$
  /* Learning, once per unroll of $T \cdot E$ = 16,384 agent steps */
  PPO on the unroll's return-phase steps only, reward r^R (skipped if fewer than 256)
    terminal at Reach or Dead; return timeout/stall bootstraps from $\gamma V_R(s_{t+1})$
  BC: 8 minibatches of 512 $(x_j, \kappa_j) \to a_j$ from demos: landmark $c \propto 1 - \text{succ}(c)$,
    skipping c with $|W_c| \geq 8$ and $\text{succ}(c) \geq 0.8$; demo uniform among c's; $j \sim \text{Unif}\{0..t_{\text{arr}}-1\}$
  Train U (RND predictor): 31 updates per 1000 agent steps, batches of 128 frames from the replay

Remark 1 (consolidation, sync and rollback only partly redirect slots). A slot's next goal ν_i is chosen when its episode ends (Alg. 4′, l. 12–18). The writes "$\nu_j \leftarrow g_m$" in consolidation, global sync and rollback therefore last only until slot j's own episode ends, which overwrites them. They take effect only for the slot that triggered the consolidation. Otherwise these operations act through g_m: it is part of the winner set of every dead slot, and it is the fallback for ineligible goals. Unlike GoWU (Alg. 1, l. 30), Alive slots that improved a landmark keep following their own frontier after a consolidation. After a rollback, the winner of a dead slot may still be a high-R landmark from I_m rather than the ancestor.

Remark 2 (where the counters change). n_g^{tgt} counts episodes at their start, after the eligibility fallback. W_g counts every return attempt, including deaths, but not the trivial handoff of Alg. 1′, l. 2. An R-improvement of g resets W_g, ℓ_g and the demos, but not n_g^{tgt}, n_g^{reach}, n_g^{hand} or n_g^{qd}.

Hyperparameters (v2)

Environment	Montezuma's Revenge, sticky actions 0.25, 18 actions, episode ends at the first life loss, cap 5000 steps
Archive	cell $(\text{level}, \text{room}, \#\text{keys}, \lfloor x/16 \rfloor, \lfloor y/16 \rfloor)$; U hysteresis 0.1; top-4 demos; 12 GB demo store
RND	Nature torso + 1024-1024 head, input-normalized, lr 3×10^{-4}, batch 128, 31 updates / 1k agent steps, replay 512k
π_R, V_R	IMPALA ResNet (16-32-32, 1024-d), separate for π_R and V_R; heads $1024{+}34 \to 512 \to 18$ / 1; bf16
PPO	$\gamma = 0.99$, $\lambda = 0.95$, lr 10^{-4}, clip 0.1, 1 epoch, 8 minibatches, entropy 10^{-3}, value 0.5, grad-norm 0.5
Return reward	1 on reaching g, 0.1 per new ancestor, $0.1\,\text{sign}(r)$
BC	alternating with PPO, 8×512 per update, cross-entropy, filter succ ≥ 0.8 after 8 attempts
Selection	$M = 4$, $N = 32$, $K \in [16, 32]$ episodes, $p_{\text{mix}} = 0.1$, rollback window 64, $\rho = 0.05$, $k \in [1, 3]$
Eligibility	unreachable gate 32 targets; doomed gate ≥ 16 handoffs with $> 90\%$ dying within 8 steps
Explore	held random action, hold $\in [5, 15]$, $\epsilon = 0.2$, budget 1000, stall 300

References

[1] Z. Mhammedi and J. Cohan. *Decoupling Exploration and Policy Optimization: Uncertainty Guided Tree Search for Hard Exploration.* arXiv:2603.22273, 2026.